Elastic Cuckoo Page Tables

Dimitrios Skarlatos, Apostolos Kokolis, Tianyin Xu, Josep Torrellas

Outline

Motivation

Elastic Cuckoo Page Tables

Evaluation

Section 1

Motivation

Address Translation is Expensive

- Page table latency is critical for memory intensive workloads where the working set is $\gg$ TLB reach

Address Translation is Expensive

- Page table latency is critical for memory intensive workloads where the working set is $\gg$ TLB reach
  - Memory intensive workloads are emerging rapidly
    - **TB-PB** of memory required

Address Translation is Expensive

- Page table latency is critical for memory intensive workloads where the working set is $\gg$ TLB reach
  - Memory intensive workloads are emerging rapidly
    - **TB-PB** of memory required
  - TLB scaling is limited by access time, space, power

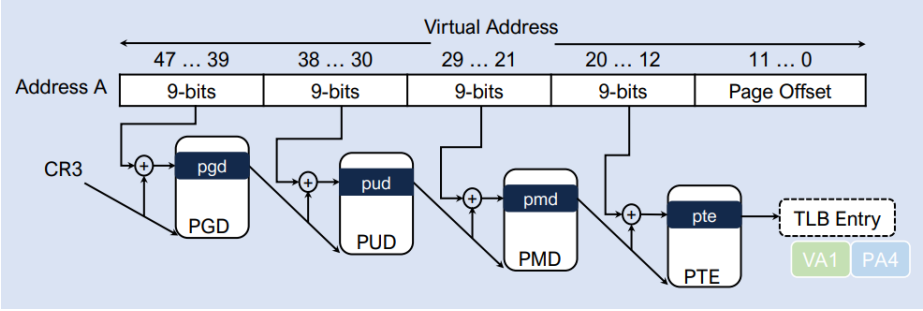
Address Translation is Expensive

- Page table latency is critical for memory intensive workloads where the working set is $\gg$ TLB reach
  - Memory intensive workloads are emerging rapidly
    - **TB-PB** of memory required
  - TLB scaling is limited by access time, space, power
- Translation can account for **20-50%** of execution time

Address Translation is Expensive

- Page table latency is critical for memory intensive workloads where the working set is $\gg$ TLB reach
  - Memory intensive workloads are emerging rapidly
    - **TB-PB** of memory required
  - TLB scaling is limited by access time, space, power
- Translation can account for **20-50%** of execution time
- Page table walks can account for **20-40%** of main memory accesses

Radix Trees



Radix Trees are the current standard for page table design.

Hashing-Based Page Tables

- Well studied alternative to radix page tables

Hashing-Based Page Tables

- Well studied alternative to radix page tables
  - Have been used commercially → IBM PowerPC, HP PA-RISC, Intel Itanium

Hashing-Based Page Tables

- Well studied alternative to radix page tables
  - Have been used commercially → IBM PowerPC, HP PA-RISC, Intel Itanium
- Address translation involves hashing the virtual page number and using the hash key to index the page table

Hashing-Based Page Tables

- Well studied alternative to radix page tables
  - Have been used commercially → IBM PowerPC, HP PA-RISC, Intel Itanium
- Address translation involves hashing the virtual page number and using the hash key to index the page table
- Loses spatial locality in page table accesses

Hashing-Based Page Tables

- Well studied alternative to radix page tables
  - Have been used commercially → IBM PowerPC, HP PA-RISC, Intel Itanium
- Address translation involves hashing the virtual page number and using the hash key to index the page table
- Loses spatial locality in page table accesses
  - Continous pages are no longer neighbors after hashing*

Hashing-Based Page Tables

- Well studied alternative to radix page tables
  - Have been used commercially → IBM PowerPC, HP PA-RISC, Intel Itanium
- Address translation involves hashing the virtual page number and using the hash key to index the page table
- Loses spatial locality in page table accesses
  - Continous pages are no longer neighbors after hashing*
- Must associate a tag with all entries → additional spatial overhead*

Hashing-Based Page Tables

- Well studied alternative to radix page tables
  - Have been used commercially → IBM PowerPC, HP PA-RISC, Intel Itanium
- Address translation involves hashing the virtual page number and using the hash key to index the page table
- Loses spatial locality in page table accesses
  - Continous pages are no longer neighbors after hashing*
- Must associate a tag with all entries → additional spatial overhead*
- **Hash collisions**

Hash Collisions

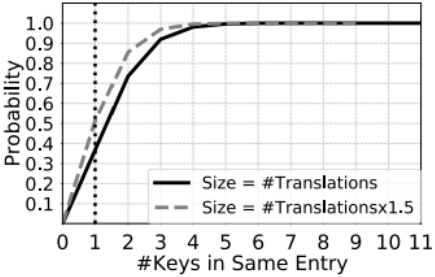
- Number of entries in the hash table must be much smaller than the size of the virtual address space

Hash Collisions

- Number of entries in the hash table must be much smaller than the size of the virtual address space
  - Resize operations are expensive

Hash Collisions

- Number of entries in the hash table must be much smaller than the size of the virtual address space
  - Resize operations are expensive
- Collisions lead to more memory accesses as the system walks the collision chain



Section 2

Elastic Cuckoo Page Tables

Cuckoo Hashing

- *Cuckoo hashing is a collision resolution algorithm that allows an element to have multiple possible hashing locations*

Cuckoo Hashing

- *Cuckoo hashing is a collision resolution algorithm that allows an element to have multiple possible hashing locations*
  - It can move between these hashing locations

Cuckoo Hashing

- *Cuckoo hashing is a collision resolution algorithm that allows an element to have multiple possible hashing locations*
  - It can move between these hashing locations
- A single cuckoo hash table contains multiple ways, indexed with separate hash functions

Cuckoo Hashing

- *Cuckoo hashing is a collision resolution algorithm that allows an element to have multiple possible hashing locations*
  - It can move between these hashing locations
- A single cuckoo hash table contains multiple ways, indexed with separate hash functions
  - n way cuckoo hash table $\rightarrow n - ary$

Cuckoo Hashing

- *Cuckoo hashing is a collision resolution algorithm that allows an element to have multiple possible hashing locations*
  - It can move between these hashing locations
- A single cuckoo hash table contains multiple ways, indexed with separate hash functions
  - n way cuckoo hash table $\rightarrow n - ary$
- Insertion places an element in one of its n possible entries

Cuckoo Hashing

- *Cuckoo hashing is a collision resolution algorithm that allows an element to have multiple possible hashing locations*
  - It can move between these hashing locations
- A single cuckoo hash table contains multiple ways, indexed with separate hash functions
  - n way cuckoo hash table $\rightarrow n - ary$
- Insertion places an element in one of its n possible entries
  - If the entry is occupied, the occupant is evicted and moved to its alternative location

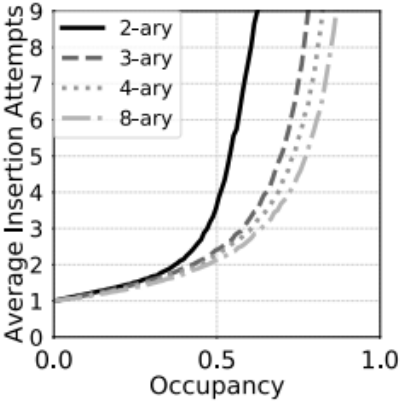
Cuckoo Hashing

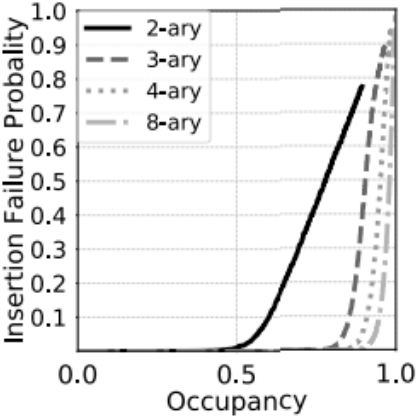
- *Cuckoo hashing is a collision resolution algorithm that allows an element to have multiple possible hashing locations*
  - It can move between these hashing locations
- A single cuckoo hash table contains multiple ways, indexed with separate hash functions
  - n way cuckoo hash table $\rightarrow n - ary$
- Insertion places an element in one of its n possible entries
  - If the entry is occupied, the occupant is evicted and moved to its alternative location
  - Repeat this until a vacancy is found

Cuckoo Hashing

- *Cuckoo hashing is a collision resolution algorithm that allows an element to have multiple possible hashing locations*
  - It can move between these hashing locations
- A single cuckoo hash table contains multiple ways, indexed with separate hash functions
  - n way cuckoo hash table $\rightarrow n - ary$
- Insertion places an element in one of its n possible entries
  - If the entry is occupied, the occupant is evicted and moved to its alternative location
  - Repeat this until a vacancy is found
- Finding an element in an $n - ary$ hash table requires only n **independent** lookups

Eviction Overhead





Rethinking Page Tables

- Default (initial) sized hashed page tables cannot be very large

Rethinking Page Tables

- Default (initial) sized hashed page tables cannot be very large
  - Forced to have modest sized, collision prone tables

Rethinking Page Tables

- Default (initial) sized hashed page tables cannot be very large
  - Forced to have modest sized, collision prone tables
  - Resizing will be necessary as insertions occur

Rethinking Page Tables

- Default (initial) sized hashed page tables cannot be very large
  - Forced to have modest sized, collision prone tables
  - Resizing will be necessary as insertions occur
- **Resizing cuckoo-hashed page tables is expensive**

Rethinking Page Tables

- Default (initial) sized hashed page tables cannot be very large
  - Forced to have modest sized, collision prone tables
  - Resizing will be necessary as insertions occur
- **Resizing cuckoo-hashed page tables is expensive**
  - Naive solution requires accessing n ways of old *and* new tables $\rightarrow$ $2 \times n$ memory accesses *per entry*

Paper Contributions

1. Extends cuckoo hashing with the new **Elastic Cuckoo Hashing** algorithm for gradual resizing

Paper Contributions

1. Extends cuckoo hashing with the new **Elastic Cuckoo Hashing** algorithm for gradual resizing
2. Builds per process **Elastic Cuckoo Page Tables**

Elastic Cuckoo Hashing

- Introduce per-way **rehashing pointers** to aid with element migration

Elastic Cuckoo Hashing

- Introduce per-way **rehashing pointers** to aid with element migration
  - Pointer dividing table into migrated region and live region

Elastic Cuckoo Hashing

- Introduce per-way **rehashing pointers** to aid with element migration
  - Pointer dividing table into migrated region and live region
  - Updated upon rehashing of any element

Elastic Cuckoo Hashing

- Introduce per-way **rehashing pointers** to aid with element migration
  - Pointer dividing table into migrated region and live region
  - Updated upon rehashing of any element
- Lookup of element only requires n probes during migration

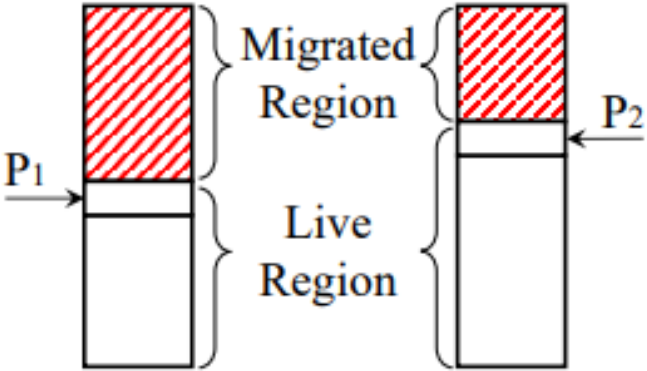
Elastic Cuckoo Hashing

- Introduce per-way **rehashing pointers** to aid with element migration
  - Pointer dividing table into migrated region and live region
  - Updated upon rehashing of any element
- Lookup of element only requires n probes during migration
  - Use hash value + pointer to determine if it resides in new or old table

Elastic Cuckoo Hashing

- Introduce per-way **rehashing pointers** to aid with element migration
  - Pointer dividing table into migrated region and live region
  - Updated upon rehashing of any element
- Lookup of element only requires n probes during migration
  - Use hash value + pointer to determine if it resides in new or old table
- Minimizes cache pollution $\rightarrow$ never fetch entries from already migrated region

Migration



Elastic Cuckoo Page Table Design

- Private, per-process page tables

Elastic Cuckoo Page Table Design

- Private, per-process page tables
  - Separate tables for different page sizes (PUD, PMD, PTE)

Elastic Cuckoo Page Table Design

- Private, per-process page tables
  - Separate tables for different page sizes (PUD, PMD, PTE)
  - Allows parallel lookups when page size is unknown
- Single hash table entry contains a VPN tag and multiple packed physical page translation entries

Elastic Cuckoo Page Table Design

- Private, per-process page tables
  - Separate tables for different page sizes (PUD, PMD, PTE)
  - Allows parallel lookups when page size is unknown
- Single hash table entry contains a VPN tag and multiple packed physical page translation entries
- Utilizes **Cuckoo Walk Tables** to further reduce number of parallel lookups

Cuckoo Walk Tables

- Software tables managed by OS to provide HW with extra context

Cuckoo Walk Tables

- Software tables managed by OS to provide HW with extra context
- Amount of context provided within CWTs results in one of four possible walk types

Cuckoo Walk Tables

- Software tables managed by OS to provide HW with extra context
- Amount of context provided within CWTs results in one of four possible walk types

1. **Complete Walk:** CWTs cannot aid in translation, HW must perform a complete lookup

Cuckoo Walk Tables

- Software tables managed by OS to provide HW with extra context
- Amount of context provided within CWTs results in one of four possible walk types

1. **Complete Walk:** CWTs cannot aid in translation, HW must perform a complete lookup
2. **Partial Walk:** CWTs can eliminate some page size(s), but cannot specify one

Cuckoo Walk Tables

- Software tables managed by OS to provide HW with extra context
- Amount of context provided within CWTs results in one of four possible walk types

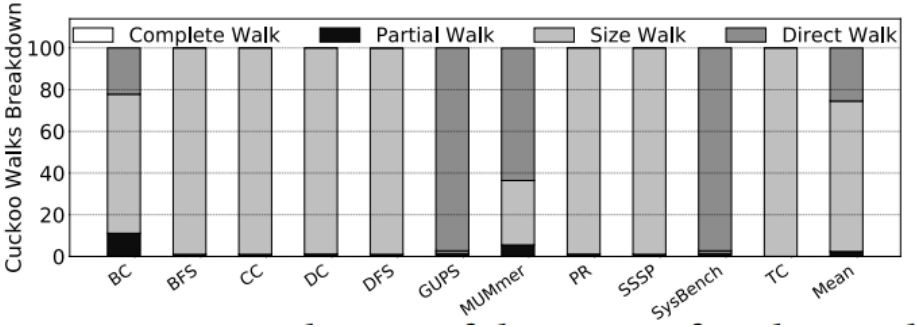
1. **Complete Walk:** CWTs cannot aid in translation, HW must perform a complete lookup
2. **Partial Walk:** CWTs can eliminate some page size(s), but cannot specify one
3. **Size Walk:** CWTs specify size of page

Cuckoo Walk Tables

- Software tables managed by OS to provide HW with extra context
- Amount of context provided within CWTs results in one of four possible walk types

1. **Complete Walk:** CWTs cannot aid in translation, HW must perform a complete lookup
2. **Partial Walk:** CWTs can eliminate some page size(s), but cannot specify one
3. **Size Walk:** CWTs specify size of page
4. **Direct Walk:** CWTs specify size and way of page

Walk Types


Complete Walk
Partial Walk
Size Walk
Direct Walk
Cuckoo Walks Breakdown
100
80
60
40
20
0
BC
BFS
CC
DC
DFS
GUPS
MUMmer
PR
SSSP
SysBench
TC
Mean

Cuckoo Walk Tables

- One CWT associated with each page size
  - PUD-CWT, PMD-CWT, PTE-CWT

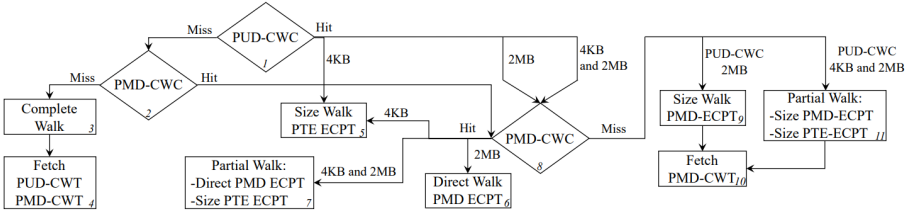
Cuckoo Walk Tables

- One CWT associated with each page size
  - PUD-CWT, PMD-CWT, PTE-CWT
- Accessed in sequence → each has the potential to provide progressively finer grained information

Cuckoo Walk Tables

- One CWT associated with each page size
  - PUD-CWT, PMD-CWT, PTE-CWT
- Accessed in sequence → each has the potential to provide progressively finer grained information
- Cached in MMU (not the PTE-CWT)

ECPT Translation Procedure



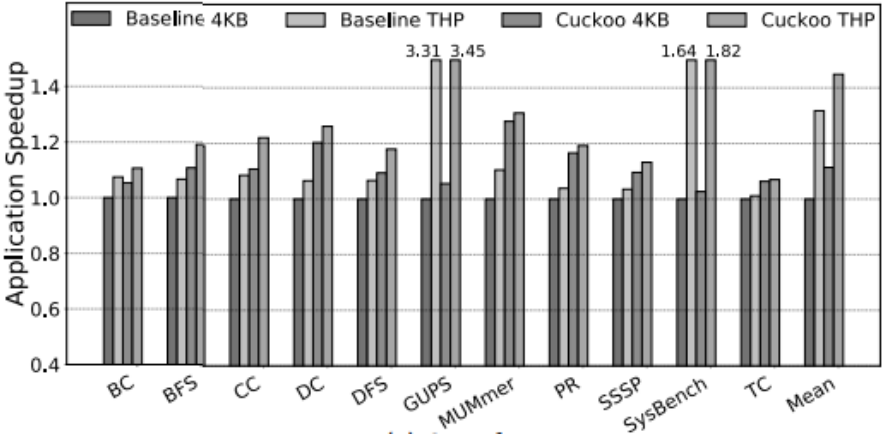
Section 3

Evaluation

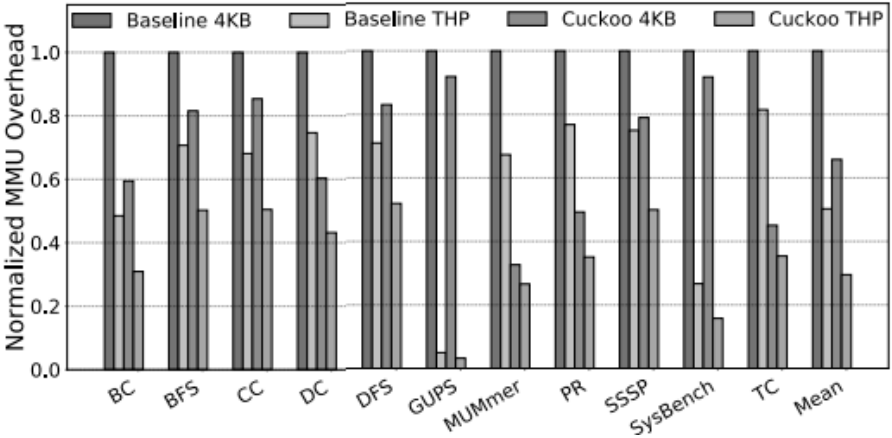
Evaluation Methodology

- Full system cycle-level simulation
- Baseline $\rightarrow$ 4-level radix page table architecture (x86-64 like)
- Proposed system evaluated using $3 - ary$ ECPTs
- Benchmark programs run with and without THP

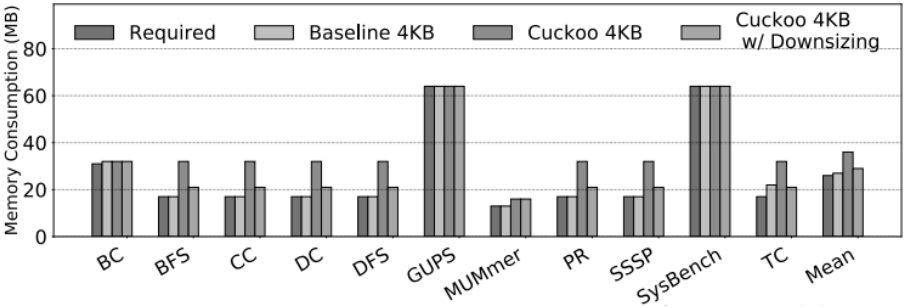
Speedup


Baseline 4KB
Baseline THP
Cuckoo 4KB
Cuckoo THP
Application Speedup
0.4
0.6
0.8
1.0
1.2
1.4
3.31
3.45
1.64
1.82
BC
BFS
CC
DC
DFS
GUPS
MUMmer
PR
SSSP
SysBench
TC
Mean

MMU Overhead Reduction



Memory Consumption



The end.